Nama : Davyn Xyello Cristwant (Alex)
Kelas : TI-1G
No. Presensi : 2

PERCOBAAN 1

1. Jika inisialisasi i = 1 diubah menjadi i = 0 maka * yg akan tercetak akan menjadi 6 * karena perulangan berjalan sebanyak 6 kali, yaitu pada i = 0, 1, 2, 3, 4, 5
2. Jika kondisi i <= n diganti menjadi i < n maka perulangan akan dijlankan sebanyak 4x untuk i = 1, 2, 3, 4 karena Operator < tidak menyertakan nilai n di dalam rentang iterasi sehingga iterasi terakhir saat i = 5 akan dilewati
3. Prediksi Output untuk n = 9: Potongan kode tersebut akan mencetak bilangan genap mulai dari 2 sampai 8 yang dipisahkan spasi, yaitu: 2 4 6 8.
4. Karena inisialisasi dimulai dari angka positif (misal $i = 1$) dan kondisi batasnya adalah $i \leq n$ (misal $n = 5$). Jika menggunakan $i--$, nilai i akan terus berkurang menjauhi nilai n (menjadi $0, -1, -2, ...$), sehingga kondisi $i \leq n$ akan selamanya bernilai benar (true) dan program tidak akan pernah berhenti.

PERCOBAAN 2

1. Variabel i berfungsi sebagai outer loop (mengatur jumlah baris), sedangkan variabel j berfungsi sebagai inner loop (mengatur jumlah kolom atau bintang per baris).
2. Outer loop dijalankan sebanyak 4 kali.
   Inner loop dijalankan secara total sebanyak $4 \times 7 = 28$ kali (karena setiap 1 baris, inner loop berputar 7 kali).
   Perintah print("*") dijalankan sebanyak 28 kali.
3. Bentuk persegi panjang tetap akan tercetak dengan dimensi yang sama (3 baris dan 6 kolom), namun alur eksekusinya terbalik di mana proses pembentukan dilakukan per kolom terlebih dahulu baru berganti baris.
4. Seluruh karakter bintang dari baris pertama hingga baris terakhir akan bercetak bergabung mendatar dalam satu baris panjang tanpa turun baris, sehingga bentuk persegi panjangnya rusak.

PERCOBAAN 3

1. Output awal tidak membentuk segitiga sama sekali, melainkan tampil lurus ke samping dan mengalami infinite loop (atau tidak berjalan sesuai keinginan karena inisruktur i dan j yang belum di-reset/diatur dengan benar). Perbaikan utama: menambahkan System.out.println() setelah inner loop agar ada perpindahan baris, serta memperbaiki kondisi logika batasan while.
2. Variabel i mengatur baris dan jumlah maksimal bintang pada baris tersebut, sedangkan j menghitung cetakan bintang per baris. Nilai j harus di-reset ke 0 pada setiap awal iterasi outer loop agar pada baris berikutnya pencetakan bintang dimulai kembali dari awal hitungan 0. Jika tidak di-reset, nilai j akan terus bertambah dan membuat baris-baris berikutnya tidak tercetak.
3. Untuk membuat segitiga terbalik, inisialisasi outer loop dimulai dari $i = n$, kondisinya diubah menjadi $i \geq 1$ (atau $i > 0$), dan perubahannya menggunakan decrement ($i--$).

PERCOBAAN 4

1. Karena totalNilai harus di-reset menjadi 0 setiap kali berpindah kelompok agar nilai dari kelompok sebelumnya tidak ikut terakumulasi ke kelompok baru. Sebaliknya, rataTertinggi tidak boleh di-reset karena variabel ini bertindak sebagai penyimpan nilai pembanding global yang harus terus mengingat rekor rata-rata tertinggi dari kelompok-kelompok sebelumnya.
2. Badan perulangan FOR dijalankan sebanyak $3 \times 2 = 6$ kali secara total (2 kali perputaran untuk masing-masing dari 3 kelompok).
3. Jika totalNilai berupa int dan dibagi dengan jumlahPenilai (yang juga int), Java akan melakukan integer division (pembagian bulat) yang membuang angka di belakang koma. Contoh: jika total nilai 155 dibagi 2 penilai, hasilnya bukan 77.5 melainkan tercetak 77.0, sehingga akurasi perhitungan rata-rata menjadi rusak.
4. Jika diberi nilai awal 0, maka jika seluruh kelompok ternyata mendapat rata-rata di bawah 0 (atau jika rentang nilai memungkinkan angka negatif), program bisa salah menentukan kelompok terbaik. Angka -1 dipilih karena berada di luar rentang nilai valid (0–100), memastikan bahwa berapapun nilai rata-rata positif pertama yang dimasukkan, itu pasti akan langsung menggantikan posisi -1.
5. WHILE (Luar): Berperan untuk mengontrol dan mengiterasi pemrosesan data berdasarkan jumlah kelompok.
   FOR (Tengah): Berperan untuk mengontrol iterasi pengambilan masukan nilai dari setiap penilai di dalam suatu kelompok.
   DO-WHILE (Dalam): Berperan untuk melakukan input validation (validasi input), yaitu memastikan agar nilai yang dimasukkan oleh penilai benar-benar berada pada rentang 0 sampai 100.